

Synthetic PDW Scenario Generator — Mini-Lab

A self-contained lab for a CS-strong intern new to the RF / radar / signals world. Everything you need to learn the theory and start coding is in this document — no external books required.

0. What this project is (30-second orientation)

A radar transmits short bursts of radio energy called **pulses**. A passive listening receiver — an **ESM** receiver (Electronic Support Measures) — does not transmit; it just *listens* to the radio spectrum and detects other people’s radar pulses. For each pulse it detects, it measures a handful of numbers (when it arrived, what frequency, how long it was, what direction it came from, how strong it was) and packs them into one record called a **PDW — Pulse Descriptor Word**.

In the real world, many radars are transmitting at once, so the receiver hears all of their pulses **mixed together in time**, plus noise. A downstream program called a **deinterleaver** (part of the **ESM back end**) has to take that jumbled stream of PDWs and sort it back into “here are the pulses from radar A, here are the pulses from radar B, ...” and estimate each radar’s parameters.

To test and tune a deinterleaver you need **realistic input with known ground truth**. That is exactly what you are building: a **Synthetic PDW Scenario Generator**. You describe a scene in a config file (“3 radars with these parameters, this much noise”), and the generator produces (a) a PDW file that looks like what the receiver would output, and (b) a truth table saying what you actually put in — so the deinterleaver’s answer can be graded against it.

You will not touch any RF hardware. This is a **pure simulation / data-generation** task: NumPy, some probability, and a clean CLI. The physics enters only as a set of equations that describe how pulse arrival times and measurements are distributed. This lab teaches you those equations and maps each one to a few lines of Python.

1. Learning objectives

By the end of this lab you will be able to:

1. **Explain the PDW model** — state what a Pulse Descriptor Word contains and why a pulse train is fundamentally a **sequence of arrival times**.
2. **Derive and implement the standard PRI patterns** — fixed, staggered, jittered, and dwell-switched (agile) pulse-repetition-interval behaviours — from their equations.
3. **Model measurement noise** as additive Gaussian perturbations on each measured parameter, and reason about the effect of the standard deviation.
4. **Interleave multiple emitters** into a single time-sorted stream (“what the receiver sees”).
5. **Corrupt the stream realistically** with Poisson-distributed false alarms (spurious pulses) and Bernoulli dropouts (missed pulses).

6. **Translate each equation into vectorised NumPy** and connect it to the project's existing `emitters.py` / `generate.py` code.
 7. **Produce, verify, and interpret** the diagnostic plots (PRI histograms, interleaved RF-vs-TOA) that an engineer uses to sanity-check a scenario.
 8. Deliver **deterministic, config-driven, tested** software — the same standard as production test-vector tooling.
-

2. Background & theory

Everything below is written for a programmer. Wherever an equation appears, **every symbol is defined immediately after it**. Units matter in this domain, so they are stated everywhere: time in **microseconds (μs)**, frequency in **megahertz (MHz)** or **gigahertz (GHz)**, angle in **degrees ($^\circ$)**.

2.1 The pulse and its parameters

A single radar pulse is a short burst of a sine wave. Four numbers describe it as the receiver sees it:

Symbol	Name	Meaning	Typical unit
t	TOA — Time Of Arrival	the instant the pulse is detected	μs
f	RF — Radio Frequency	the carrier frequency of the sine wave	MHz
τ	PW — Pulse Width	how long the burst lasts	μs
θ	AoA — Angle of Arrival	the bearing to the emitter	degrees
A	amplitude	received signal strength	(relative / dB)

Intuition for a CS major. Think of RF as *which channel* the pulse is on, PW as *the duration of the blip*, AoA as *the compass bearing to whoever sent it*, and amplitude as *how loud it was*. TOA is simply *the timestamp*.

A **PDW** is the tuple of these measured values for one pulse:

$$\text{PDW} = (t, f, \tau, \theta, A).$$

In this project the on-disk column order (fixed by `pdw.py`, **do not change it**) is:

TOA_us RF_MHz AoA_deg PW_us emitter_id

`emitter_id` is a bookkeeping label the *generator* knows (ground truth). A real receiver does **not** get told the id — recovering it is the deinterleaver's whole job. We carry it so we can grade the deinterleaver later.

2.2 A pulse train is a sequence of arrival times

A single radar fires pulses over and over. Ignoring RF/PW/AoA for a moment, the *timing* of one radar is completely described by the ordered set of arrival times

$$T = \{t_0, t_1, t_2, \dots, t_{N-1}\}, t_0 < t_1 < \dots < t_{N-1},$$

where

- t_n = TOA of the n -th pulse (μs),
- N = number of pulses in the observation window.

The gap between consecutive pulses is the **PRI — Pulse Repetition Interval**:

$$\text{PRI}_n = T_n = t_n - t_{n-1} (\mu\text{s}).$$

Its reciprocal is the **PRF — Pulse Repetition Frequency**, the number of pulses per second:

$$\text{PRF} = \frac{1}{\bar{T}}, \bar{T} = \text{mean PRI (in seconds)}.$$

So $\bar{T} = 1000 \mu\text{s}$ (i.e. 10^{-3}s) corresponds to $\text{PRF} = 1000$ pulses/s.

The entire timing behaviour of a radar is a rule for generating the sequence $\{t_n\}$. Different radars use different rules — and those rules are precisely what a deinterleaver keys on to tell emitters apart. The next five subsections are those rules. This is the mathematical core of the whole project, so read them slowly.

Why do radars bother varying the PRI at all? Two reasons worth knowing (you do not need to model the physics, just the *pattern*): (1) **Range de-ambiguation** — a constant PRI makes distant targets alias to the wrong range; staggering/switching the PRI breaks that ambiguity. (2) **Anti-jam / low-probability-of-intercept** — an irregular PRI is harder for a listener to predict and track. Ironically, that is exactly what *you* are simulating so the deinterleaver can learn to cope with it.

2.3 Fixed PRI (the constant-rate radar)

The simplest radar fires at a perfectly constant interval T . Its arrival times are an arithmetic progression:

$$t_n = t_0 + nT, n = 0, 1, 2, \dots$$

- t_n = TOA of pulse n (μs),
- t_0 = time of the first pulse — the **start phase** (μs),
- T = the PRI, a constant (μs),
- n = pulse index (integer).

Why randomise t_0 ? The absolute time when we start observing is arbitrary — the radar was already running before we tuned in. Modelling that means the first pulse can land anywhere within one PRI of $t = 0$, uniformly:

$$t_0 \sim U(0, T),$$

where $U(a, b)$ denotes the continuous **uniform distribution** on the interval $[a, b]$ — every value in the range is equally likely. Using a random start phase (rather than always $t_0=0$) keeps multiple emitters from all lining up at the origin, which would be an unrealistic artefact.

This is the **worked example** already implemented for you in `emitters.py`:

```
class FixedPRI(Emitter):
    def toas(self, t_end_us, rng):
        t0 = rng.uniform(0, self.pri)           # t0 ~ U(0, T)
        return np.arange(t0, t_end_us, self.pri) # t0, t0+T,
t0+2T, ...
```

`np.arange(t0, t_end, T)` is the equation $t_n = t_0 + nT$: it emits `t0`, `t0+T`, `t0+2T`, ... stopping before `t_end`. `rng` is a NumPy random generator (`np.random.default_rng(seed)`); passing it around — rather than calling `np.random.*` globally — is what makes runs **reproducible** given a seed (an acceptance requirement).

2.4 Staggered PRI (a fixed repeating pattern)

A **staggered** radar cycles through a short, fixed list of M intervals and repeats it forever. For example the pattern $\{T_1, T_2, T_3\} = \{466, 566, 466\} \mu s$ gives gaps 466, 566, 466, 466, 566, 466, ... The intervals are the ordered pattern

$$\{T_1, T_2, \dots, T_M\},$$

- T_k = the k -th interval in the pattern (μs), for $k=1, \dots, M$,
- M = pattern length (the number of distinct steps before it repeats).

The n -th arrival time is the start phase plus the sum of the first n intervals, where the interval index **wraps around** the pattern using modular arithmetic:

$$t_n = t_0 + \sum_{k=0}^{n-1} T_{(k \bmod M)+1}, n=0, 1, 2, \dots$$

- t_n, t_0, n — as before,
- $k \bmod M$ = the remainder of k divided by M — this makes the pattern index cycle $0, 1, \dots, M-1, 0, 1, \dots$; the $+1$ shifts it to the 1-based labels $T_1 \dots T_M$,
- the empty sum (when $n=0$) is 0, so t_0 is just the start phase.

In words: **cumulatively sum the cycled pattern**. Each gap is one entry of the pattern, taken in order and looping when you run off the end. In code this is `cumsum` over a tiled/looped pattern:

```
def stagger_toas(pattern, t_end, rng):
    pattern = np.asarray(pattern, dtype=float)   # [T_1, ..., T_M]
    M = len(pattern)
    t0 = rng.uniform(0, pattern[0])              # random start phase
    # how many intervals do we need to (over)fill the window?
    n_int = int(np.ceil(t_end / pattern.mean())) + M
```

```

    gaps = pattern[np.arange(n_int) % M]           #  $T_{\{(k \bmod M)+1\}}$ ,
    k=0..n_int-1
    toas = t0 + np.concatenate([[0.0], np.cumsum(gaps)]) # prepend t0
    itself
    return toas[toas < t_end]

```

`np.arange(n_int) % M` is the literal $k \bmod M$; indexing `pattern[...]` with it realises $T_{(k \bmod M)+1}$; `np.cumsum` performs the $\sum$. Note we **prepend a 0** before the cumulative sum so that the very first arrival is t_0 itself (the $n=0$ empty-sum case).

This is the `StaggerPRI.toas` you must implement. Its test only requires that the observed gaps be exactly the pattern values — see §5.

2.5 Jittered PRI (a noisy radar)

A **jittered** radar aims for a mean PRI $\bar{T}$ but each interval is randomly perturbed — oscillator noise, deliberate dither, etc. Model each interval as the mean plus zero-mean Gaussian noise:

$$T_n = \bar{T} + \eta_n, \eta_n \sim N(0, \sigma_T^2).$$

- T_n = the n -th interval (μs),
- $\bar{T}$ = the **mean PRI** (μs),
- η_n = the random jitter added to interval n (μs),
- $N(\mu, \sigma^2)$ = the **normal (Gaussian) distribution** with mean μ and variance σ^2 ; here $\mu=0$,
- σ_T = the **PRI jitter standard deviation** (μs) — the “wobble” amount.

The arrival times are then the running total of the intervals:

$$t_n = t_0 + \sum_{k=1}^n T_k.$$

Jitter is often quoted as a **fraction** of the mean rather than an absolute number. If the **fractional jitter** is j (e.g. $j=0.05$ means “5 % jitter”), then

$$\sigma_T = j\bar{T}.$$

Two subtleties you must handle in code:

1. **A PRI can never be ≤ 0 .** A large negative draw of η_n could make $T_n \leq 0$, which is physically impossible (pulses can’t arrive before the previous one). Clip to a small positive floor: `intervals = np.clip(intervals, eps, None)`.
2. **Draw enough intervals.** You don’t know N in advance; over-generate intervals to safely fill the window, cumulative-sum, then clip to $[0, t_{end})$.

```

def jitter_toas(mean, jitter, t_end, rng):
    n_guess = int(t_end / mean * 1.5) + 10           # over-generate to
    fill the window
    intervals = rng.normal(mean, jitter, n_guess)    #  $T_n = \text{mean} +$ 
    eta_n, eta_n ~  $N(0, \text{jitter}^2)$ 
    intervals = np.clip(intervals, 1e-6, None)      # PRI must stay > 0

```

```

t0 = rng.uniform(0, mean)                                # start phase
toas = t0 + np.cumsum(intervals)                          # t_n = t0 +
sum_{k<=n} T_k
return toas[toas < t_end]

```

`rng.normal(mean, jitter, n)` draws n samples of $N(\bar{T}, \sigma_T^2)$ (NumPy's second argument is the **standard deviation**, not the variance). Because the errors accumulate through the `cumsum`, the *absolute* timing drifts more and more over the window even though each *interval* stays close to $\bar{T}$ — that random walk is exactly the real behaviour.

This is the `JitterPRI.toas` you must implement.

2.6 Dwell-switched / agile PRI (piecewise-constant)

A **dwell-switched** (a.k.a. **agile** or **PRI-switching**) radar holds one PRI constant for a block of time — a **dwell** — then abruptly switches to the next PRI, cycling through a list. The PRI as a function of time is a **step function** (piecewise-constant):

$$T(t) = T^{(b)} \text{ for } t \in [t_b, t_b + D),$$

- $T^{(b)}$ = the PRI used during block b (μs), taken from a cyclic list $\{T^{(0)}, T^{(1)}, \dots\}$,
- D = the **dwell time** — how long each block lasts before switching (μs),
- t_b = the start time of block b ; block $b+1$ begins where block b 's dwell elapses.

Within a block the arrivals are just fixed-PRI at $T^{(b)}$; when the elapsed time in the current block reaches D , you advance to the next PRI. The simplest correct implementation walks pulse by pulse:

```

def dwell_toas(pris, dwell, t_end, rng):
    pris = list(pris)                                # [T^(0), T^(1), ...]
    t = rng.uniform(0, pris[0])                      # start phase
    out, block_start, b = [], t, 0
    while t < t_end:
        out.append(t)
        t += pris[b % len(pris)]                    # step by the CURRENT
    block's PRI
        if t - block_start >= dwell:                 # this dwell block is over
    -> switch
        block_start = t
        b += 1
    out = np.asarray(out)
    return out[out < t_end]

```

The two ideas to get right: **(a)** step by the *current* block's PRI, and **(b)** when the time you've spent in the block reaches `dwell`, reset the block clock and move to the next PRI. On a PRI histogram this looks like a few discrete peaks (like stagger) — the difference is *temporal*: stagger interleaves its intervals pulse-by-pulse, whereas dwell uses one PRI for a long run before switching. A plot of PRI-vs-time makes the blocks obvious.

This is the `DwellPRI.toas` you must implement.

2.7 (Bonus) Sinusoidal PRI modulation

Some radars sweep the PRI smoothly rather than in steps. A common model modulates the interval sinusoidally around the mean:

$$T_n = \bar{T} + A \sin(2\pi f_m n \bar{T}).$$

- T_n = the n -th interval (μs),
- $\bar{T}$ = mean PRI (μs),
- A = **modulation amplitude** — the peak deviation of the PRI from the mean (μs),
- f_m = **modulation frequency** — how fast the PRI is swept (Hz); note $n\bar{T}$ is the approximate elapsed time (so its product with f_m is dimensionless if $\bar{T}$ is in seconds — keep units consistent),
- n = pulse index.

```
def sinusoid_toas(mean, amp, fm_hz, t_end, rng):
    n = np.arange(int(t_end / mean * 1.5) + 10)
    mean_s = mean * 1e-6 # us -> s so f_m
    * t is unitless
    intervals = mean + amp * np.sin(2*np.pi*fm_hz * n * mean_s)
    intervals = np.clip(intervals, 1e-6, None)
    toas = rng.uniform(0, mean) + np.cumsum(intervals)
    return toas[toas < t_end]
```

This is an optional stretch pattern — implement it only after the four required ones pass.

2.8 Measurement noise (per-parameter Gaussian error)

So far we handled *timing*. The receiver also *measures* RF, PW, and AoA, and every real measurement is imperfect. Model each measured value as the true value plus independent, zero-mean Gaussian noise:

$$\hat{x} = x + v, v \sim N(0, \sigma_x^2), x \in \{\text{RF}, \text{PW}, \text{AoA}\}.$$

- x = the true parameter value (whatever the emitter actually transmits),
- $\hat{x}$ = the **measured** (noisy) value that goes into the PDW — the “hat” means “estimate of”,
- v = the measurement error for this pulse,
- σ_x = the measurement **standard deviation** for parameter x (its own units: MHz for RF, μs for PW, degrees for AoA).

Each parameter gets its **own** σ because receivers measure frequency, duration, and bearing with different precisions. Larger σ = a fuzzier measurement = harder for the deinterleaver to cluster pulses by RF/PW/AoA. This is **already implemented** in the base `Emitter.pulses`:

```
def pulses(self, t_end_us, rng):
    t = np.asarray(self.toas(t_end_us, rng), dtype=float) # timing
    from the subclass
    n = len(t)
```

```

    rf = self.rf + rng.normal(0, self.rf_jit, n) # RF_hat = RF
+ N(0, rf_jit^2)
    pw = self.pw + rng.normal(0, self.pw_jit, n) # PW_hat = PW
+ N(0, pw_jit^2)
    aoa = self.aoa + rng.normal(0, self.aoa_jit, n) # AoA_hat = AoA
+ N(0, aoa_jit^2)
    return np.column_stack([t, rf, aoa, pw, np.full(n, self.id)])

```

Each line is one instance of $\hat{x} = x + v$. Notice the constructor's `rf_jit`, `pw_jit`, `aoa_jit` are exactly the σ_x values, with sensible defaults. You don't have to write this — but you must **understand** it, because your scenario configs set these knobs, and your lab report must explain what raising them does to the stream.

2.9 Interleaving (what the receiver actually hears)

A scene has several emitters, each producing its own set of pulses P_i (a P_i is that emitter's list of PDW rows). The receiver hears the **union of all of them, sorted by time of arrival**:

$$P = \text{sort}_{\text{TOA}} \left(\bigcup_i P_i \right)$$

- P_i = the set of pulses from emitter i ,
- $\bigcup_i$ = the **union** over all emitters (concatenate every emitter's pulses into one pile),
- $\text{sort}_{\text{TOA}}(\cdot)$ = sort the combined pile by the TOA column,
- P = the final interleaved stream the receiver outputs.

This is why the problem is hard: pulses from A, B, C arrive **shuffled together in time**, and the labels are stripped. In code, “union” is a vertical stack and “sort by TOA” is an argsort on column 0 — which `pdw.write_pdw` already does for you on write:

```

pulses = np.vstack(all_pulses) # union: one pile of
rows
pulses = pulses[np.argsort(pulses[:, 0])] # sort by TOA (column
0)

```

`generate.py` builds `all_pulses` as a list of each emitter's `pulses(...)` array; `np.vstack` is the $\bigcup$, and the sort happens inside `write_pdw`. (Figure 3 visualises this union as three coloured RF layers.)

2.10 Spurious pulses / false alarms (Poisson process)

Real receivers occasionally report pulses that **no emitter sent** — thermal noise spikes, multipath, distant clutter. These are **false alarms** (a.k.a. spurious pulses). Because they arise from many independent rare events, their *count* in a fixed window follows a **Poisson distribution**:

$$N_{fa} \sim \text{Poisson}(\lambda W).$$

- N_{fa} = the number of false-alarm pulses generated in the window,
- λ = the **false-alarm rate** (false alarms per unit time — e.g. per second, or per μs ; keep units consistent with W),

- W = the length of the observation window (same time unit as λ),
- $\text{Poisson}(\mu)$ = the Poisson distribution with mean μ ; it returns non-negative integers and models “number of independent rare events in an interval”, with $E[N_{fa}] = \lambda W$.

Each false alarm is then placed **uniformly at random in time**, with RF and PW drawn uniformly across the scene’s parameter ranges (a false alarm carries no real emitter’s signature):

$$t_j^{fa} \sim U(0, W), f_j^{fa} \sim U(f_{min}, f_{max}), \tau_j^{fa} \sim U(\tau_{min}, \tau_{max}).$$

```
def inject_spurious(rate_per_us, t_end, rf_range, pw_range, rng):
    n_fa = rng.poisson(rate_per_us * t_end)           #  $N_{fa} \sim$ 
    Poisson(lambda * W)
    toa = rng.uniform(0, t_end, n_fa)                #  $t \sim U(0, W)$ 
    rf = rng.uniform(*rf_range, n_fa)                #  $RF \sim U(f_{min},$ 
    f_max)
    pw = rng.uniform(*pw_range, n_fa)                #  $PW \sim U(pw_{min},$ 
    pw_max)
    aoa = rng.uniform(-180, 180, n_fa)              # bearing
    anywhere
    eid = np.zeros(n_fa)                             #  $id = 0 \Rightarrow$  "not
    a real emitter"
    return np.column_stack([toa, rf, aoa, pw, eid])
```

Note on the config knob. The project config uses `spurious_frac` — a *fraction of the real pulse count* — rather than an absolute rate λ . That is a convenient reparameterisation: pick $N_{fa} = \text{round}(\text{spurious}_{\frac{\times}{N_{real}}})$ directly, or convert to a rate via $\lambda = N_{fa} / W$. Both are valid; the Poisson form above is the underlying physical model, and either interpretation is acceptable for your deliverable. Give the false alarms `emitter_id = 0` so they’re identifiable as ground-truth “not real”.

2.11 Dropout (Bernoulli thinning)

The opposite failure: the receiver **misses** real pulses (it was busy, the pulse was too weak, two pulses overlapped). Model this as **Bernoulli thinning** — keep each real pulse independently with probability $1 - p_d$, drop it with probability p_d :

$$\text{keep pulse } n \Leftrightarrow u_n > p_d, u_n \sim U(0, 1).$$

- p_d = the **dropout probability** (per pulse), $0 \leq p_d \leq 1$,
- u_n = an independent uniform draw for pulse n ; if it exceeds p_d the pulse survives,
- “thinning” = randomly deleting points from a point process; here each point (pulse) survives with probability $1 - p_d$.

The expected number of survivors from N pulses is $N(1 - p_d)$.

```
def apply_dropout(pulses, p_d, rng):
    keep = rng.random(len(pulses)) > p_d           #  $u_n > p_d \Rightarrow$ 
```

```

survive
    return pulses[keep]

```

One vectorised line: draw one uniform per pulse, keep the rows whose draw beats p_d . (Figure 5 shows survivors, drops, and false alarms together.)

2.12 Putting the pipeline together

The full generation pipeline — most of which `generate.py` already wires up — is:

1. **Build** emitters from the config (`build(cfg)`).
2. For each emitter, generate its **timing** $\{t_n\}$ (the PRI rule) and attach **noisy** RF/PW/AoA (§2.8) → its pulse set P_i .
3. **Interleave**: stack all P_i and sort by TOA (§2.9).
4. **Corrupt**: inject Poisson false alarms (§2.10) and apply Bernoulli dropout (§2.11).
5. **Write** the PDW file (time-sorted) and the truth table.

Determinism comes from threading a single seeded rng through every random step. Your two TODOs in `generate.py` are steps 4a and 4b.

3. Worked examples

Short, **runnable** snippets that implement each equation and state the expected output. Each is a self-contained `python3 -c '...'` you can paste into a terminal from the project folder, or drop into a scratch file. They deliberately mirror the code you'll write in `emitters.py` / `generate.py`.

3.1 Fixed PRI — constant spacing

```

import numpy as np
rng = np.random.default_rng(0)
T = 500.0                                # PRI = 500 us
t0 = rng.uniform(0, T)                   # start phase ~ U(0, T)
t = np.arange(t0, 5000, T)               # t_n = t0 + nT over a 5
ms window
print("first 5 TOAs:", np.round(t[:5], 1))
print("all gaps == 500?", np.allclose(np.diff(t), 500))

```

Expected output: the gaps are all exactly 500, e.g.

```

first 5 TOAs: [ 318.5  818.5 1318.5 1818.5 2318.5]
all gaps == 500? True

```

(The exact start phase depends only on the seed; the gaps are always 500.)

3.2 Staggered PRI — the pattern appears in the gaps

```

import numpy as np
rng = np.random.default_rng(0)

```

```

pattern = np.array([400.0, 500.0, 650.0])    # M = 3 intervals
M = len(pattern)
t0 = rng.uniform(0, pattern[0])
n_int = int(np.ceil(5000 / pattern.mean())) + M
gaps = pattern[np.arange(n_int) % M]        #  $T_{\{(k \bmod M)+1\}}$ 
t = t0 + np.concatenate([[0.0], np.cumsum(gaps)])
t = t[t < 5000]
print("unique rounded gaps:", sorted({float(x) for x in
np.round(np.diff(t))}))

```

Expected output: only the three pattern values appear as gaps:

```
unique rounded gaps: [400.0, 500.0, 650.0]
```

3.3 Jittered PRI — mean interval $\approx$ mean PRI

```

import numpy as np
rng = np.random.default_rng(0)
mean, jitter = 500.0, 50.0                #  $T_n \sim N(500, 50^2)$ 
intervals = np.clip(rng.normal(mean, jitter, 2000), 1e-6, None)
t = rng.uniform(0, mean) + np.cumsum(intervals)
d = np.diff(t)
print("mean gap :", round(d.mean(), 1), "(expect ~500)")
print("gap stddev:", round(d.std(), 1), "(expect ~50)")

```

Expected output (numbers vary slightly with seed, tolerances are loose):

```
mean gap : 498.6 (expect ~500)
gap stddev: 50.0 (expect ~50)
```

3.4 Dwell-switched PRI — both PRIs show up, in blocks

```

import numpy as np
rng = np.random.default_rng(0)
pris, dwell = [600.0, 700.0], 6000.0
t, out, block_start, b = rng.uniform(0, pris[0]), [], None, 0
block_start = t
while t < 80000:
    out.append(t)
    t += pris[b % len(pris)]
    if t - block_start >= dwell:
        block_start = t; b += 1
d = np.round(np.diff(np.sort(np.array(out))))
print("PRIs present:", sorted(set(d) & {600.0, 700.0}), "(expect both)")

```

Expected output:

```
PRIs present: [600.0, 700.0] (expect both)
```

3.5 Measurement noise — noisy RF scatters around truth

```
import numpy as np
rng = np.random.default_rng(0)
rf_true, sigma = 9200.0, 4.0 # MHz
rf_hat = rf_true + rng.normal(0, sigma, 1000) #  $x_{\text{hat}} = x + nu$ ,  $nu \sim N(0, \sigma^2)$ 
print("mean(rf_hat):", round(rf_hat.mean(), 2), "(expect ~9200)")
print("std (rf_hat):", round(rf_hat.std(), 2), "(expect ~4)")
```

Expected output:

```
mean(rf_hat): 9199.81 (expect ~9200)
std (rf_hat): 3.91 (expect ~4)
```

3.6 Interleaving — union then sort

```
import numpy as np
rng = np.random.default_rng(0)
# two toy emitters: (toa, rf) only, ids 1 and 2
A = np.column_stack([np.arange(0, 3000, 500.0), np.full(6, 9200),
np.ones(6)])
B = np.column_stack([np.arange(0, 3000, 700.0), np.full(5, 2900),
2*np.ones(5)])
both = np.vstack([A, B]) # union
both = both[np.argsort(both[:, 0])] # sort by TOA (col 0)
print("interleaved ids by time:", both[:, 2].astype(int))
print("TOAs sorted?", np.all(np.diff(both[:, 0]) >= 0))
```

Expected output: the id column alternates as the two trains interleave, and TOAs are non-decreasing:

```
interleaved ids by time: [1 2 1 2 1 2 1 1]
TOAs sorted? True
```

3.7 Poisson false alarms + Bernoulli dropout

```
import numpy as np
rng = np.random.default_rng(0)
W = 80000.0 # 80 ms window
# --- Poisson false alarms ---
lam = 200e-6 # 200 false alarms / second
n_fa = rng.poisson(lam * W) #  $N_{\text{fa}} \sim \text{Poisson}(\lambda W)$ 
print("false alarms:", n_fa, "(expect ~", round(lam*W), ")")
# --- Bernoulli dropout on a 1000-pulse train ---
p_d = 0.1
kept = (rng.random(1000) > p_d).sum()
print("kept after dropout:", kept, "(expect ~900)")
```

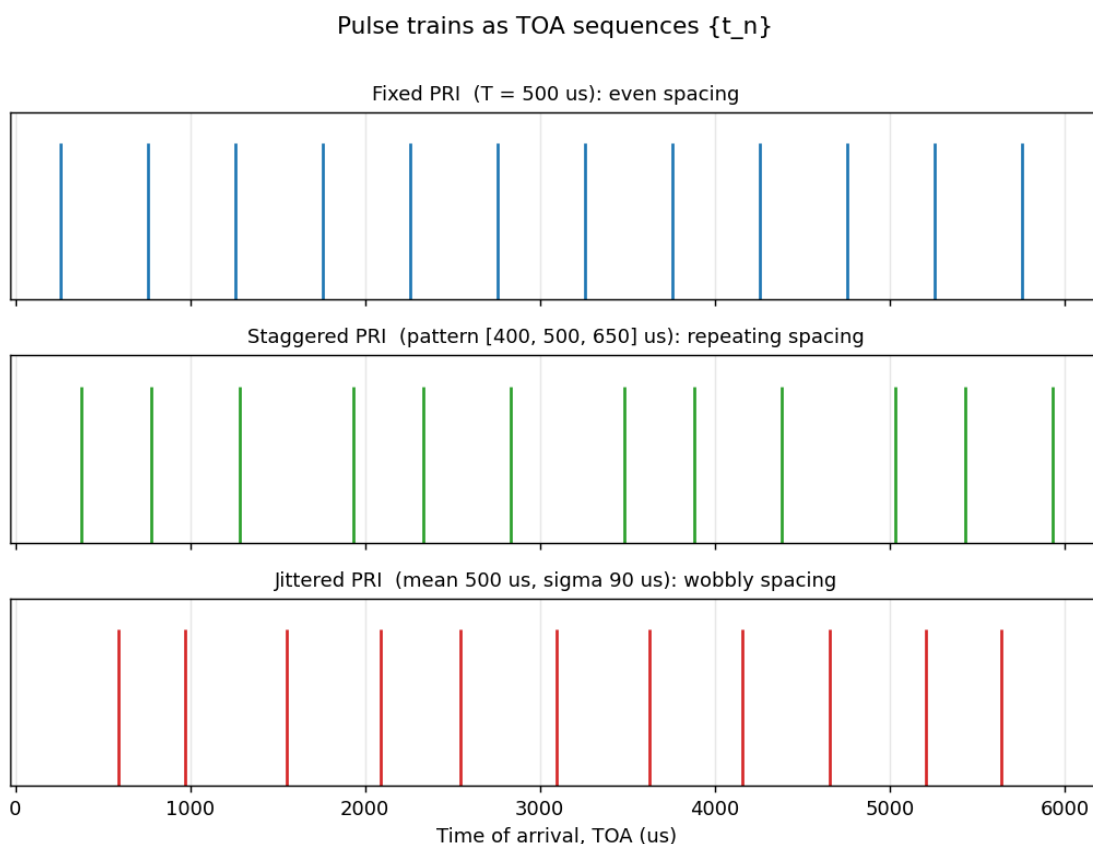
Expected output (Poisson/Bernoulli counts fluctuate around their means):

false alarms: 18 (expect ~ 16)
kept after dropout: 910 (expect ~900)

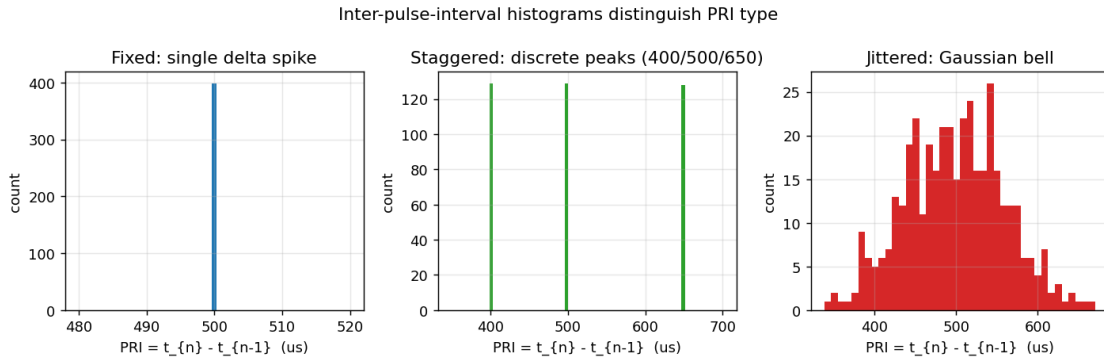
How these map to your code: 3.2/3.3/3.4 are the bodies of StaggerPRI/JitterPRI/DwellPRI.toas. 3.7's two halves are the two TODOs in generate.py. 3.5/3.6 are already done in Emitter.pulses and write_pdw — reproduced so you understand what the framework does for you.

4. Figures

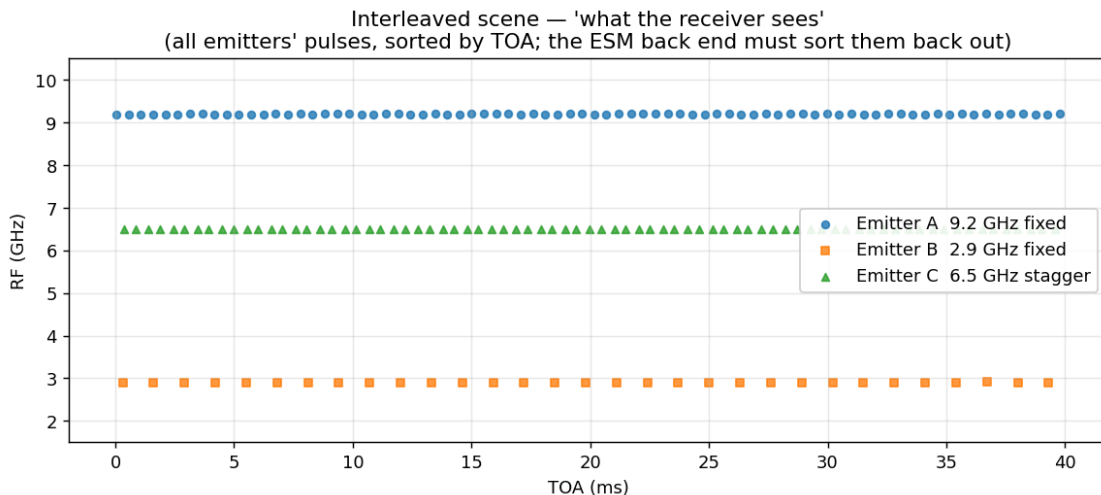
All figures are produced by `figs/make_figs.py` (run `python3 figs/make_figs.py`). That script reimplements the TOA generators in a few lines each, so it doubles as a compact reference for the math → code mapping. **Your own implementations in `emitters.py` should reproduce the same shapes.**



*Fixed, staggered, and jittered pulse trains shown as TOA stem plots over a 6 ms window. Each vertical line is one pulse arrival. **Top (fixed):** perfectly even spacing — the arithmetic progression $t_n = t_0 + nT$. **Middle (staggered):** a repeating short-long-... rhythm as the pattern $[400, 500, 650] \mu s$ cycles. **Bottom (jittered):** roughly-periodic but visibly irregular spacing, because each interval is $\bar{T}$ plus Gaussian noise. This one picture is the whole idea of §2.3–2.5: a radar's identity lives in the rule that spaces its pulses.*

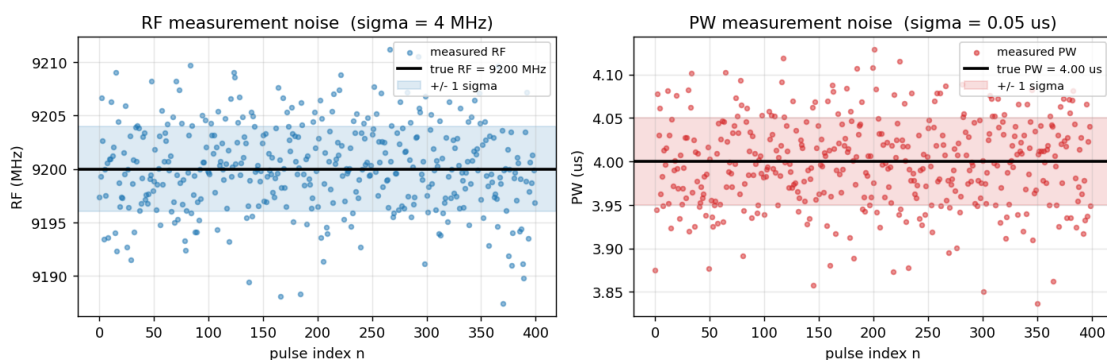


Histograms of the inter-pulse interval ($PRI = t_n - t_{n-1}$) for the three PRI types, built from a long (200 ms) run so the shapes are clear. **Left (fixed):** a single sharp spike at 500 μs — every gap is identical. **Middle (staggered):** three discrete peaks at exactly 400/500/650 μs — the pattern values, and nothing in between. **Right (jittered):** a Gaussian bell centred on the mean PRI, its width set by σ_T . The PRI histogram is the single most useful diagnostic for “what kind of emitter is this?”, and reproducing these three shapes from your generator is a required deliverable.

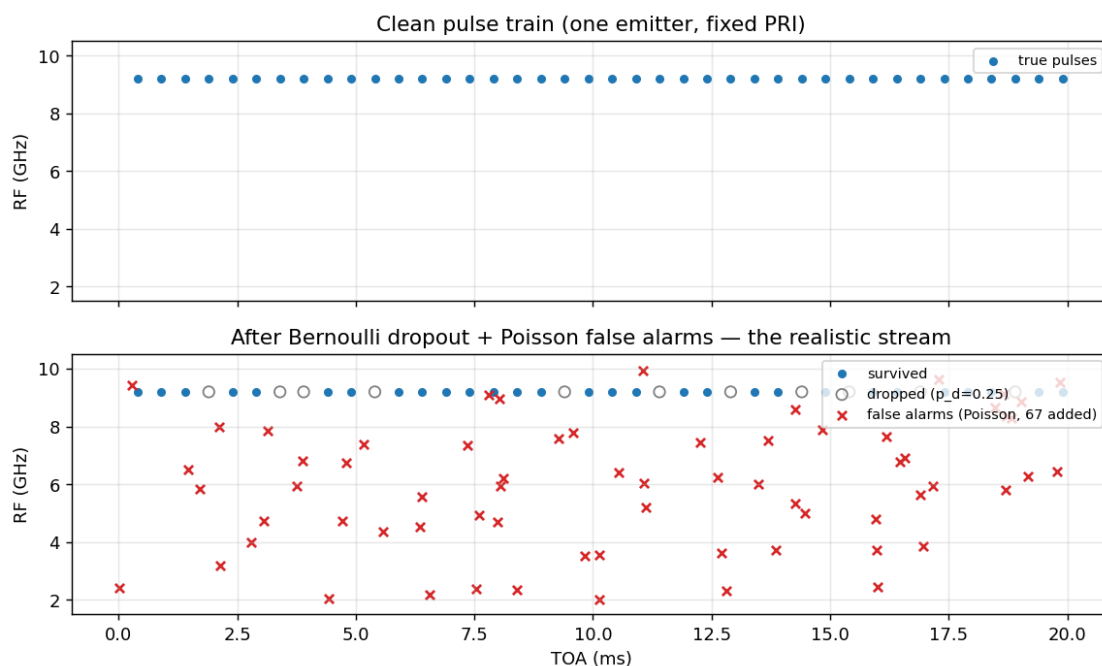


An interleaved three-emitter scene: measured RF (GHz) on the y-axis versus TOA (ms) on the x-axis, each emitter drawn in its own colour/marker. This is literally “what the receiver sees” — the union of all emitters’ pulses, time-sorted (§2.9) — except here we cheat by colouring the true source. In the real PDW file those colours are gone and every row is just numbers; the ESM back end (the deinterleaver) has to recover the grouping. Notice each emitter forms a horizontal band at its RF (with slight scatter from measurement noise), which is exactly the structure a clustering deinterleaver exploits.

Per-parameter measurement noise: $\hat{x} = x + \nu$, $\nu \sim N(0, \sigma_x^2)$



Measurement noise for a single emitter: 400 pulses' measured RF (left, in MHz) and PW (right, in μ s) scattered around their true values (solid black line), with the $\pm 1\sigma$ band shaded. Each point is one pulse's $\hat{x} = x + \nu$, $\nu \sim N(0, \sigma_x^2)$ (§2.8). About two-thirds of the points fall inside the shaded band — the defining property of a Gaussian. RF and PW use different σ values because a receiver measures frequency and duration with different precisions. Raising σ smears these bands wider and makes the emitter harder to isolate.



The effect of spurious pulses and dropout. **Top:** a clean fixed-PRI train from one emitter — a neat row of evenly-spaced points at 9.2 GHz. **Bottom:** the same train after (a) Bernoulli dropout — surviving pulses stay filled blue, dropped ones are shown as hollow grey circles where a pulse should have been (§2.11), and (b) Poisson false alarms — red $\times$'s scattered uniformly in time and across the RF band (§2.10). The bottom panel is what a realistic PDW file looks like: gaps where real pulses were missed, plus a haze of junk that was never transmitted. Coping with both is the deinterleaver's job, and generating both is your second `generate.py` TODO.

5. Your deliverables

Hand-in checklist. Check every box. “Pass” always means `pytest -q` is green **with no skips**.

- ❑ **Implement StaggerPRI, JitterPRI, DwellPRI** in `emitters.py`, using the equations in §2.4 / §2.5 / §2.6 respectively (do **not** modify `FixedPRI`, `pdw.py`, or the writer column order). Follow the `FixedPRI` shape: implement only each class’s `toas(self, t_end_us, rng)`.
- ❑ **Un-skip and pass their tests** in `tests/test_emitters.py` — delete the three `@pytest.mark.skip(...)` decorators on `test_stagger_pattern`, `test_jitter_mean`, `test_dwell_switches`, and make them pass. (Read each test first: e.g. stagger requires the observed gaps to be a subset of the pattern; jitter requires the mean interval within 20 μ s of the mean PRI; dwell requires both PRIs to appear.)
- ❑ **Implement Poisson spurious-pulse injection** in `generate.py` (first TODO): add N_{fa} extra rows (from `spurious_frac`, per §2.10) with uniform-random TOA in $[0, t_{end}]$, RF/PW drawn across the scene’s ranges, `emitter_id = 0`. Use the shared `rng`.
- ❑ **Implement Bernoulli dropout** in `generate.py` (second TODO): delete a fraction `dropout_frac` of the pulses via thinning (§2.11), using the shared `rng`.
- ❑ **Author ≥ 5 scenario configs** under `configs/` (in addition to `example_scene.yaml`): vary emitter count, PRI types (include at least one each of fixed/stagger/jitter/dwell across your set), RF/PW/AoA, measurement-noise levels, and spurious/dropout fractions. Each must run: `python3 generate.py configs/<yours>.yaml`.
- ❑ **From YOUR generator’s output**, produce and save as PNGs: (a) a **PRI histogram per emitter type** (one panel per PRI behaviour, like Figure 2 but built from your `scene_pdw.txt`), and (b) an **interleaved RF-vs-TOA plot** of a multi-emitter scene (like Figure 3). You may adapt `figs/make_figs.py` or write a small `analyze.py` that reads a PDW file — but the data must come from your generator, not from the reference generators in `make_figs.py`.
- ❑ **Write a 1–2 page lab report** (`LAB_REPORT.md`, skeleton below) covering: each PRI type with its equation; your PRI histograms and interleaved plot; and how measurement noise, spurious false alarms, and dropout each change the stream.
- ❑ **Determinism check**: running the same config with the same seed twice produces byte-identical `scene_pdw.txt`. (`python3 generate.py cfg.yaml && cp scene_pdw.txt a.txt && python3 generate.py cfg.yaml && diff a.txt scene_pdw.txt` prints nothing.)

5.1 LAB_REPORT.md skeleton (copy this into a new file and fill it in)

Lab Report – Synthetic PDW Scenario Generator

Author: <your name> Date: <date>

1. PRI models implemented

For each of fixed / staggered / jittered / dwell:

- the governing equation (copy from the lab, in your own words),
- how you implemented ``toas()`` (1–2 sentences),
- the test that proves it works.

2. PRI histograms (from my generator)

Embed your per-type PRI histogram PNG(s). One sentence each: what shape do you see and why (delta / discrete peaks / bell / blocks)?

3. Interleaved multi-emitter scene

Embed your RF-vs-TOA plot. How many emitters, what types, what does the receiver "see"? Point out the horizontal RF bands.

4. Corruptions

- Measurement noise: what happens to the RF/PW bands as you raise sigma?
- Spurious (Poisson): how many false alarms did you add, and where do they land?
- Dropout (Bernoulli): what fraction did you drop; how does the train look after?

5. Determinism & config-driven design

Show that same seed => identical output, and that adding an emitter is a config-only change.

6. What I'd do next (stretch, optional)

e.g. sinusoidal PRI, moving emitters, scan-modulated amplitude.

6. Milestones

Planned at roughly **16 hr/week**. Adjust to your pace; the *order* matters more than the exact hours. "Green" always means `pytest -q` passes with no skips.

Week 1 — Orient & fixed baseline (≈16 hr). Read this whole lab. Skim `README.md`, `pdw.py`, `emitters.py`, `generate.py`, `tests/`. Run the worked example: `python3 generate.py configs/example_scene.yaml`, inspect `scene_pdw.txt` / `scene_truth.txt`. Run `python3 figs/make_figs.py` and open all 5 PNGs. Run the §3 worked snippets and confirm the expected outputs. Trace how `FixedPRI.toas` implements $t_n = t_0 + nT$. *Deliverable:* a paragraph in your report explaining the PDW format and the fixed-PRI equation.

Week 2 — Stagger + Jitter (≈16 hr). Implement `StaggerPRI.toas` (§2.4) and `JitterPRI.toas` (§2.5). Un-skip `test_stagger_pattern` and `test_jitter_mean`; iterate until green. Add a scenario config that uses a stagger and a jitter emitter and confirm it generates. Sanity-check by plotting their PRI histograms from your output (expect discrete peaks and a bell). *Deliverable:* two classes green + one new config.

Week 3 — Dwell + corruptions (≈16 hr). Implement `DwellPRI.toas` (§2.6); un-skip `test_dwell_switches`; green. Implement the two `generate.py` TODOs: Poisson spurious injection (§2.10) and Bernoulli dropout (§2.11). Verify determinism (same seed => identical file).

Author the remaining configs to reach **≥ 5 total**, covering all four PRI types and a range of noise/spurious/dropout settings. *Deliverable*: all tests green, both corruptions working, **≥ 5** configs.

Week 4 — Analysis, plots, report (≈16 hr). From *your* generator’s output, produce the per-type PRI histogram and the interleaved RF-vs-TOA plot (adapt `make_figs.py` or write `analyze.py` that reads a PDW file). Write `LAB_REPORT.md`. Final polish: re-run `pytest -q` (green, no skips), re-check determinism, tidy configs. *Optional stretch if time*: sinusoidal PRI (§2.7), a moving emitter, or a reusable emitter-template library. *Deliverable*: the finished report + plots + green test suite.

7. Acceptance criteria

Your submission is accepted when **all** of the following hold:

1. **Tests green, no skips.** `pytest -q` passes; the three previously-skipped tests (`test_stagger_pattern`, `test_jitter_mean`, `test_dwell_switches`) are unskipped and pass, and the fixed-PRI tests still pass.
 2. **All four PRI types implemented** per their equations (§2.3–2.6), each demonstrably producing the correct PRI-histogram signature (delta / discrete peaks / bell / blocks).
 3. **Corruptions implemented.** `generate.py` injects Poisson false alarms (`id = 0`, uniform TOA, RF/PW over the scene range) and applies Bernoulli dropout, both driven by the config fractions and the shared `rng`.
 4. **Determinism.** Same config + same seed $\Rightarrow$ **byte-identical** `scene_pdw.txt` on repeat runs.
 5. **Config-driven extensibility.** A new emitter can be added by editing a YAML config only — no code change. Adding a config file requires no edits to existing files.
 6. **Output format unchanged.** `pdw.py`’s columns/order (TOA RF AoA PW id) and the truth-table format are untouched; downstream tools still parse the file.
 7. **≥ 5 scenario configs** exist under `configs/` and each generates without error.
 8. **Deliverable plots** exist and are built **from your generator’s output**: a per-type PRI histogram and an interleaved RF-vs-TOA plot, saved as PNGs.
 9. **Lab report** (`LAB_REPORT.md`, 1–2 pages) present and complete per the skeleton in §5.1.
-

8. Reading (free / online)

All free, all online — no paywalled books required.

- **Radar Tutorial** — clear, diagram-heavy primer on radar basics. Start with PRI/PRF and pulse parameters. <https://www.radartutorial.eu/> (see the “Radar Basics” book, Chapter 1).
- **MIT OpenCourseWare** — *Introduction to Radar Systems* (RES.LL-001 / the Lincoln Laboratory course). Free lecture notes/videos; skim the introduction and the waveforms/pulse lectures for the vocabulary. <https://ocw.mit.edu/> (search “Introduction to Radar Systems”).

- **PySDR: A Guide to SDR and DSP using Python** — friendly, code-first introduction to sampling, signals, and noise in Python/NumPy; excellent for building signal intuition as a programmer. <https://pysdr.org/>
 - **Wikipedia** — **“Pulse repetition frequency”** — concise coverage of PRF/PRI, staggered PRF, and range ambiguity. https://en.wikipedia.org/wiki/Pulse_repetition_frequency
 - **Wikipedia** — **“Radar signal characteristics”** — defines pulse width, PRI/PRF, duty cycle, and related pulse-train terms. https://en.wikipedia.org/wiki/Radar_signal_characteristics
 - **NumPy documentation** — you’ll live in these: random generators (`numpy.random.Generator` — `.uniform`, `.normal`, `.poisson`, `.random`), plus `numpy.cumsum`, `numpy.diff`, `numpy.arange`, `numpy.column_stack`, `numpy.vstack`, `numpy.argsort`. <https://numpy.org/doc/stable/>
 - (Optional, background) **Wikipedia** — **“Poisson point process”** and **“Point process”** for the probability model behind false alarms and thinning. https://en.wikipedia.org/wiki/Poisson_point_process
-

End of mini-lab. Build the generator, keep it deterministic, and let the PRI histograms tell you whether each emitter is doing what its equation says.